

Resilience-oriented CPPS: A Simulation and Modelling procedure

By: *Alejandro Barrero*

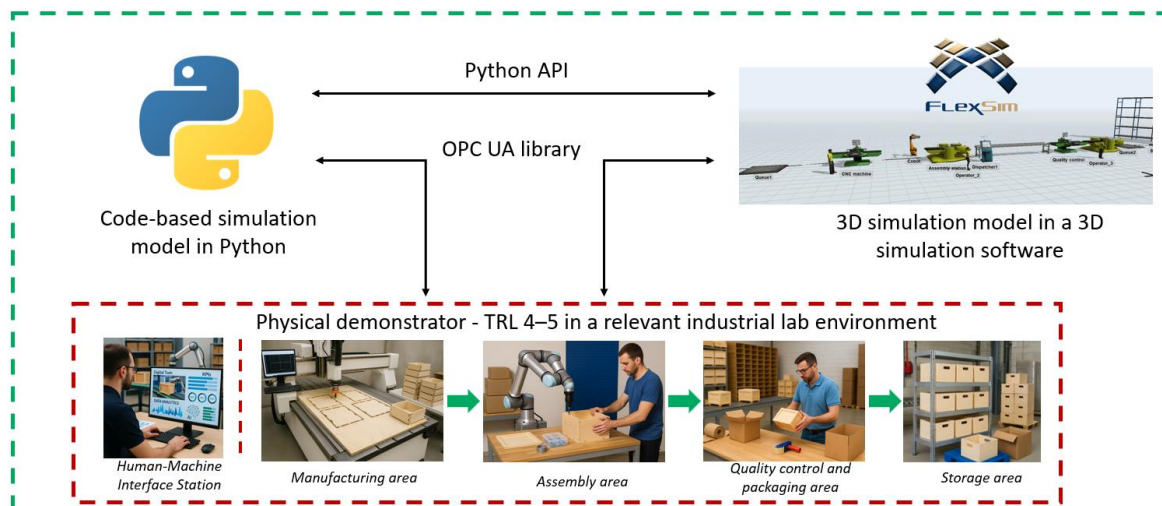
1. Problem definition

The case study focuses on the design and evaluation of a resilience-oriented CPPS for the production and assembly of a wooden box, as illustrated in the provided environment. The production of wooden boxes is a multi-stage process that requires both precision and coordination between machines, human operators, and digital control systems. In the case study under analysis, a Cyber-Physical Production System (CPPS) is designed to support the manufacturing, assembly, quality control, and storage of boxes while ensuring resilience to disruptions. The manufacturing process begins with raw wooden panels that are processed by a CNC machine, where each panel is cut into the required components: base, sides, and lid. Once machined, the parts are transferred to the assembly area, where a collaborative robot and a human operator work together to assemble the box. The robot handles repetitive or ergonomically challenging tasks, while the operator supervises alignment, applies adhesives, and ensures quality of fit.

After assembly, the box is moved to the quality control and packaging station. Here, an operator inspects dimensions, surface finish, and structural integrity before packaging the final product for delivery. The packaged boxes are then placed into the storage area, awaiting shipment. Throughout this process, a Human–Machine Interface (HMI) station monitors operations in real time, gathering data on key performance indicators such as cycle time, throughput, and resilience metrics. Overall, the system includes four interconnected areas:

- Manufacturing area (CNC machine): processes raw wooden sheets into components.
- Assembly area (Collaborative workstation): human–robot collaboration for assembling parts.
- Quality control & packaging area: operator inspects, packages, and prepares boxes.
- Storage area: packaged boxes stored until delivery.

Figure 1. CPPS environment



However, as in real-world operations, the system is exposed to disruptions. These include delays in the arrival of raw materials, breakdowns in the CNC machine or robot, operator errors during assembly or packaging, and communication failures between machines and the HMI. This case study develops a discrete-event system simulation in which the models of interest are analyzed numerically. The objective of the simulation study is therefore to evaluate how the CPPS can anticipate, resist, recover from, and adapt to such disruptions, ensuring robustness, recoverability, and adaptability across all production stages. Develop, simulate, and validate a CPPS that demonstrates resilience against disruptions is made by integrating:

- A code-based simulation model in Python (to handle stochastic parameters, event logic, and resilience KPIs).
- A 3D simulation model in FlexSim (to capture physical layouts, process times, and operator–robot interaction).

2. System parameters and data

To accurately represent the wooden box production system, the simulation requires a diverse set of parameters. These include process times, resource availability, disruption probabilities, and resilience-related performance indicators.

The CNC machining process is characterized by its cutting cycle time, which depends on the thickness and quality of the raw wooden panels. The collaborative assembly workstation requires parameters for both the robotic arm (speed, precision, and error rate) and the human operator (working pace, learning curve, and likelihood of errors). The packaging and quality control station requires inspection times and packaging cycle times, while the storage system depends on rack capacity and handling time per unit.

Resource parameters are equally important. The system must define the number of CNC machines, robots, operators, and storage racks, as well as their operational reliability, quantified through metrics such as mean time between failures (MTBF) and mean time to repair (MTTR). Operator variability must be incorporated by modeling error rates or reduced efficiency under fatigue.

Disruption parameters are critical to resilience assessment. For example, raw material delays are modeled with probability distributions reflecting supplier variability. Equipment breakdowns for CNC and robots follow stochastic failure models, while operator error rates are represented probabilistically. Communication failures in the HMI system may be modeled as interruptions in signal transmission or delays in executing control commands.

Resilience metrics serve as output measures of system performance under disruptions. These include throughput (measured in units per hour), cycle time per box, system availability, recovery time after disruptions, and adaptability—defined as the system’s ability to reconfigure workflows or reroute tasks when part of the system becomes unavailable.

3. Formulation of the simulation model

The conceptual model represents the CPPS as a discrete-event system where entities, resources, and events interact dynamically. Entities flow through a sequence of processes: first as wooden panels, then as assembled boxes, and finally as packaged products in storage. Resources include the CNC machine, collaborative robot, human operator, packaging station, and storage racks. Events represent the arrival of raw materials, execution of machining cycles, assembly operations, packaging, and placement into storage. Disruptions are treated as stochastic events that interrupt or slow down operations.

Mathematically, the system is defined using stochastic distributions to capture variability. For instance, machining times may follow a normal distribution around a mean cycle time, while machine breakdowns follow an exponential distribution reflecting time-to-failure. Operator errors are modeled as Bernoulli trials with a small probability of occurrence, and communication delays may follow uniform distributions. Resilience metrics are computed by comparing system performance under disruption with baseline conditions. For example, robustness is measured as the ratio of disrupted throughput to nominal throughput, and recovery time is defined as the time interval between disruption occurrence and the restoration of normal performance.

A hybrid simulation approach integrates two complementary environments. In Python, a code-based model (e.g., with SimPy) handles discrete events, stochastic disruptions, and KPI calculation. In FlexSim, the 3D model visualizes material flows, machine interactions, and operator–robot collaboration. Data exchange between the two environments ensures consistency: Python provides

disruption scenarios and resilience calculations, while FlexSim supplies process flows and spatial validation.

4. Conceptual model validation

Validation of the conceptual model ensures that it accurately represents the real-world wooden box production system before implementation. Expert validation is the first step: engineers, operators, and production managers review assumptions about process flow, resource allocation, and disruption scenarios. Walk-through validation is performed by presenting the conceptual flow of entities, raw material entry, CNC machining, collaborative assembly, packaging, and storage, and ensuring that it aligns with observed industrial practices.

Cross-checking with historical data strengthens the model's credibility. For instance, expected throughput values are compared with actual production logs to verify reasonableness. Little's Law ($WIP = \text{throughput} \times \text{cycle time}$) is applied to approximate system behavior under nominal conditions, and results are compared with model assumptions. Sensitivity analysis is performed conceptually, identifying parameters likely to affect resilience most strongly, such as breakdown probabilities or operator error rates. This step ensures that the model captures the right drivers of system performance.

Following Sargent's framework, the validation process also documents assumptions and limitations. For example, the assumption that packaging time is independent of operator fatigue may be challenged during review. Documenting such issues allows stakeholders to make informed judgments about the model's credibility.

5. Programming of a theoretical model and verification

The simulation model is implemented in two parallel environments. Python, using a discrete-event simulation framework such as SimPy, provides the backbone for handling events, queues, and resilience KPI calculations. It allows precise control of stochastic processes, such as equipment breakdowns or raw material delays, and facilitates the execution of multiple replications for statistical reliability. FlexSim complements this by modeling the physical layout and flow of materials in a 3D environment. Its visual interface supports operator training and facilitates validation by enabling stakeholders to "see" the process in action.

Verification activities ensure that the model is free of logical and coding errors. In Python, event logic and resource allocation are checked through unit testing and trace analysis of event sequences. In FlexSim, debugging tools are used to step through process animations, verifying that material handling follows the intended workflow. Simple test cases, such as running the system without disruptions, are compared with hand calculations of expected throughput. Any discrepancies are addressed before proceeding to experimentation.

The implementation of the simulation requires not only creating models in Python and FlexSim separately, but also establishing a bidirectional communication channel between them so they can exchange data in real time. This hybrid integration combines the flexibility and analytical power of Python (stochastic events, resilience KPIs, scenario generation) with the visualization and discrete-event flow modeling of FlexSim.

Interconnection Strategy

The Python model and the FlexSim model are linked through communication protocols and APIs that allow them to exchange variables and states dynamically. One of the most common approaches is to use OPC UA (Open Platform Communications Unified Architecture) and Python API, which provides a standard for interoperability between industrial software platforms.

- FlexSim can be configured as an OPC UA server or client, exposing model variables such as machine status, queue lengths, or throughput.

- Python can use libraries such as *OPC UA* or *FreeOpcUa* to connect to the OPC UA server, read process values, and inject disruption events or control commands.

This enables, for example:

- A breakdown event generated in Python to immediately pause the CNC machine in the FlexSim model.
- Real-time performance data (throughput, WIP levels) from FlexSim to be collected in Python for KPI calculation.
- Adaptive control strategies (e.g., rerouting, adding resources) to be computed in Python and then sent back to the FlexSim model for execution.

Verification occurs at multiple levels. First, each model is verified independently. In Python, event scheduling, failure injection, and KPI calculations are checked through unit tests and debug logs. In FlexSim, animation traces and model debugging confirm correct flows and synchronization of operators, robots, and machines. Once verified individually, integration verification ensures proper communication between the two platforms. Controlled experiments, such as forcing a disruption in Python and observing its reflection in FlexSim, confirm that data exchange is functioning. Likewise, monitoring throughput in FlexSim and comparing it with Python-collected values verifies consistency of data sharing.

This interconnected architecture effectively creates a digital twin of the CPPS, where the Python model acts as a “decision intelligence layer,” providing resilience-oriented control and analytics, while FlexSim supplies the 3D execution layer.

The following sections will be completed with the results of the simulation.

6. Tests runs
7. Operational validation
8. Simulation experiments
9. Analysis of the simulation experiments
10. Results

References

- Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (various eds.). Discrete-Event System Simulation. Prentice Hall / Pearson. (Classic text that lays out the end-to-end process of simulation studies.)
- Law, A. M. (5th ed., 2014). Simulation Modeling and Analysis. McGraw-Hill. (Formerly Law & Kelton in earlier editions; the book is the go-to on input modeling, statistical analysis, and experiment design.)
- Kritzinger, W., et al. (2018). “Digital Twin in manufacturing: A categorical literature review and classification.” IFAC-PapersOnLine.
- Negri, E., Fumagalli, L., & Macchi, M. (2017). “A review of the roles of Digital Twin in CPS-based production systems.” Procedia Manufacturing
- Sargent, R. G. (2012). “Verification and validation of simulation models,” Journal of Simulation. (Journal version of the V&V framework.)
- Brailsford, S. C., et al. (2019). “Hybrid simulation modelling in operational research,” European Journal of Operational Research. (Scholarly treatment of hybrid DES/ABM/SD.)